הפקולטה להנדסת חשמל ומחשבים
המעבדה לבקרה רובוטיקה ולמידה חישובית

הנושא:

רחפן עזר למשימות חילוץ והצלה

מגישים:

רחלי דור רז פיזאן

מנחה:

אורן רוזן

סמסטר: חורף תשפ"ו

שנה: 2026

1. Introduction

1.1 Background

Natural disasters, such as earthquakes, floods, and landslides, as well as incidents involving missing persons in challenging terrains, present critical challenges to emergency response teams. In search and rescue (SAR) operations, time is the single most vital factor; the probability of finding a survivor alive decreases exponentially with every passing hour - a concept often referred to as the "Golden Hour."

Traditionally, SAR missions rely heavily on ground personnel and manned aircraft. However, ground searches are slow, intensive and often expose rescue teams to significant physical risks in unstable environments. While the introduction of Unmanned Aerial Vehicles (UAVs) has improved accessibility, current operations still largely depend on manual piloting and manual visual analysis of the video feed. This reliance on human operators introduces the risk of fatigue and cognitive overload, potentially leading to missed detections of survivors.

1.2 The Problem

Despite the technological advancements in UAVs, several critical gaps remain in their effective deployment for SAR missions:

- Inefficiency of Manual Analysis: In a typical scenario, a drone pilot or an analyst must stare at a live video feed for hours. The human eye struggles to identify small targets (such as a person lying on the ground) from high altitudes, especially in complex environments.

- Data Overload: A drone covering a large area generates a vast amount of visual data. Processing this data manually is a bottleneck that delays the deployment of rescue teams to the correct coordinates.

- Hardware and Training Limitations: Developing and testing autonomous SAR algorithms on physical drones is expensive, resource-intensive, and at times, even dangerous, making it difficult to train off-the-shelf AI models effectively.

1.3 Project Goals

The overarching objective of this initiative was to design a fully autonomous Unmanned Aerial Vehicle (UAV) system capable of performing real-world Search and Rescue missions. The vision was to deploy a physical drone into a disaster zone that could independently navigate complex terrain, capture visual data, and identify human survivors without any human pilot intervention. The aim was to create a "force multiplier" for rescue teams - a tool that could cover vast areas faster and safer than ground personnel.

1.3.2 Realization via Simulation (Proof of Concept)

While the long-term vision involves physical deployment, this project focuses on the algorithmic and logical validation of such a system using a high-fidelity simulation. Due to hardware limitations of available off-the-shelf drones (specifically the lack of programmable onboard cameras and sufficient battery endurance for long-range missions), the project scope was defined to implement this workflow within the Microsoft AirSim environment.

Therefore, the specific goals of this project are:

- **Validate the feasibility** of autonomous aerial scanning using a "Store-and-Process" architecture.
- **Generate a synthetic dataset** that mimics real-world aerial scenarios to train Deep Learning models.
- **Demonstrate the detection capabilities** of the YOLO algorithm on aerial imagery, serving as a robust Proof of Concept for future physical implementation.

The primary objective of this project is to design and implement an automated simulation-based workflow for aerial search and rescue. The project aims to bypass the limitations of physical hardware and manual analysis by leveraging a high-fidelity virtual environment and Deep Learning.

Zooming in on the operational steps required to reach our objective:

- Simulation Environment Setup: Utilize Microsoft AirSim and Unreal Engine to create a realistic disaster zone environment. This allows for safe, repeatable, and cost-effective testing of flight patterns and camera configurations.

- Autonomous Data Acquisition: Implement an autonomous flight algorithm that systematically scans a defined area and captures high-resolution imagery without human intervention.

- AI-Based Detection: Train and fine-tune a YOLO (You Only Look Once) neural network on this synthetic data to accurately detect survivors in the captured imagery.
- Offline Processing Pipeline: Demonstrate a complete workflow where the drone acts as a data collector, and the processing unit analyzes the batch of images post-flight to output the precise locations of identified survivors.

1.4 Proposed Solution

This project proposes a system where the UAV operates as an intelligent data acquisition node. Instead of relying on real-time edge computing (which requires heavy hardware), the system focuses on a robust Store-and-Process architecture.

The workflow begins in the AirSim simulator, where a drone executes a pre-planned autonomous path over a disaster zone. During the flight, the drone captures images at calculated intervals to ensure full terrain coverage. These images are then transferred to a processing unit where a custom-trained YOLOv8 model analyzes them. The system filters empty frames and flags images containing survivors, providing rescue teams with visual confirmation and location data. This approach minimizes the risk to human searchers and significantly accelerates the identification process.

2. System Evolution and Implementation Challenges

2.1 The Original Project Vision

The overarching goal of the project was the development of an autonomous system based on a micro-drone, capable of scanning a designated terrain and identifying human figures in real-time. The initial concept relied on using a commercial, compact drone platform, with the aspiration of creating a portable and cost-effective solution. The premise was that integrating available off-the-shelf hardware with advanced Computer Vision algorithms would enable reliable and rapid detection.

2.2 Hardware Limitations and the Transition to Simulation

During the initial implementation phase, the Crazyflie 2.0 platform was evaluated. Throughout empirical testing, we encountered significant technological barriers that prevented the use of this platform for the intended mission:

- Energy Constraints: The effective flight time of the drone was limited to mere seconds or a few minutes, a duration insufficient for performing continuous scanning and image processing tasks.

- Visual Sensor Quality: The Crazyflie's built-in camera did not meet the threshold requirements (resolution and frame rate) necessary for modern object detection models.

Considering these constraints, a strategic decision was made to migrate the development and testing environment to a high-fidelity physical simulator – AirSim. This transition allowed us to neutralize hardware noise and energy limitations, enabling a focus on algorithmic development in a controlled and reproducible environment.

2.3 Challenges in the Simulation Environment

The transition to AirSim introduced new challenges regarding the mastery of the development environment. Since the software and its underlying engine were new to us, a significant learning curve was required to effectively utilize the tools.

- Environment Setup and Object Placement: Configuring the simulation and manually placing human assets within the virtual terrain requires substantial time and effort. We had to familiarize ourselves with the simulator interface and capabilities to position objects in a way that simulated diverse scenarios.

- Data Collection: Once the environment was set up, we established a workflow for capturing images from the drone at various angles and altitudes to build a rich dataset for testing.

2.4 The Domain Gap and Model Adaptation

In the analysis phase, we utilized the YOLOv8s (You Only Look Once) model, which was pre-trained on standard datasets (such as NOMAD, a specialized dataset designed for SAR research) containing real-world images. During initial tests in the simulation, the model failed to identify virtual figures as "humans." This failure is attributed to the Domain Gap phenomenon - the visual discrepancy between the realistic images on which the model was trained, and the artificial textures and lighting of the simulation.

2.5 Resolution: Retraining and Fine-tuning

To bridge this gap, a customized retraining process was required:

- Custom Dataset Generation: We extracted a repository of images from the simulation containing virtual figures (about 300 images).

- Labeling: We performed a process of creating precise labels for the images in the YOLO format.

- Model Training: We executed a Fine-tuning process for the YOLOv8s model based on the new data. This process "taught" the neural network to generalize the features of synthetic figures and identify them as people with a high level of accuracy.

2.6 Summary of the Final Outcome

The final product is a system integrating a drone within a High-Fidelity simulation environment, coupled with a Computer Vision model that has undergone specific optimization for this domain. The system successfully demonstrates the feasibility of the solution, bypassing the hardware limitations that characterized the project's inception, and provides a robust platform for future research in the field of autonomous drones.

3. Project Phases Derivation

3.1 Hardware Evaluation and Establishment of Simulation Infrastructure

This stage involved two main sub-processes. Initially, we focused on the Crazyflie platform learning the existing toolset, executing commands, and manual flight testing. Following the identification of hardware limitations (battery life and camera quality), the focus shifted to establishing a high-fidelity simulation environment using AirSim.

This phase was crucial and resource intensive. As we transitioned to AirSim, a complex software environment previously unknown to us, a significant portion of this stage was dedicated to mastering the simulator's interface, configuring the virtual terrain, and establishing a workflow for data acquisition.

3.2 Study of Object Detection Algorithms and Neural Networks

The objective was to identify the most suitable architecture for human detection within our specific constraints. We evaluated existing state-of-the-art models to determine the optimal balance between accuracy and inference speed, ultimately selecting YOLOv8s.

During this stage, we evaluated existing solutions, including Roboflow and different YOLO architectures, ultimately selecting the architecture that best suited our project requirements.

3.3 Model Integration, Training, and Domain Adaptation

This stage focused on deploying the YOLOv8s model and adapting it to our specific data distribution. While the model was pre-trained on real-world data, it failed to generalize the simulation environment due to the Domain Gap. To address the performance disparity, we moved beyond standard implementation to a process of **Fine-tuning**. This involved generating a custom synthetic dataset within AirSim, creating appropriate labels, and retraining the network. The goal was to align the model's weights with the visual characteristics of the virtual environment, ensuring high detection rates for the simulated human figures.

4. The Simulation Environment – AirSim

4.1 Software Overview: Microsoft AirSim

To address the hardware limitations encountered in the preliminary phase, the project migrated to Microsoft AirSim (Aerial Informatics and Robotics Simulation). Built upon Unreal Engine, AirSim is a high-fidelity simulator that provides a photorealistic 3D environment. It accurately simulates the physics of quadcopter flight—including dynamics, gravity, and air resistance—while mimicking the data streams of various onboard sensors such as RGB cameras. The selection of AirSim was driven by its support for "Software-in-the-Loop" (SITL) simulation, enabling the development of control algorithms in Python that can be easily ported to real-world hardware.

4.2 Environment Customization and Dataset Generation

To train the detection model effectively, we constructed a custom virtual environment designed to replicate complex terrain. Using the Unreal Engine Editor, we modified the simulation map to create a diverse testing ground:

- Asset Integration: We imported various 3D human models into the scene.
- Strategic Placement: To challenge the computer vision algorithm, figures were manually placed in various postures (standing, sitting) and locations. Some were fully exposed, while others were partially occluded by obstacles or shadows to simulate real-world conditions.
- Environmental Clutter: Inanimate objects were added to test the model's ability to distinguish between human targets and background noise.

4.3 Autonomous Navigation and Control Logic

The drone's autonomous operation was implemented using a custom Python script to interface with the AirSim API. The control logic was designed to perform a systematic Grid Scan (Lawnmower Pattern) over a defined area of interest.

The system operates based on the following high-level logic:

1. Mission Setup: The drone initiates the sequence by arming the motors and ascending to a fixed operational altitude.

2. Path Planning: The algorithm calculates a "Zigzag" flight path to ensure complete coverage of the terrain. The drone navigates autonomously from point to point, covering the entire grid efficiently.
3. Data Acquisition: At each defined waypoint along the path, the drone stabilizes and triggers the camera sensor. The system captures high-resolution RGB images and automatically logs the drone's position (telemetry data) to match each image. This automated process allowed for the rapid generation of a large, consistent dataset for the subsequent training phase.

5. Theoretical Background: Deep Learning and Object Detection

5.1 Neural Networks

The field of Neural Networks is based on the ability to analyze data, identify patterns, and draw statistical conclusions, mimicking human learning processes. The network consists of "neurons" (nodes) connected by weighted edges. Information (in our case, an image) enters through the Input Layer, undergoes processing through Hidden Layers that perform mathematical operations (multiplication and summation), and finally produces a result at the Output Layer.

In this project, we address a problem that combines Classification - determining if the object is a human or background, and Regression - finding the exact coordinates of the person in the image.

5.2 Convolutional Neural Networks (CNN)

Standard neural networks struggle to process images due to the vast amount of information contained in every pixel. The solution is Convolutional Neural Networks (CNNs), which specialize in spatial data processing.

A CNN is composed of several key layer types:

- Convolution Layer: This layer scans the image using filters or kernels. Each filter learns to identify a different visual feature - ranging from simple edges and corners in the initial layers to complex shapes like limbs or faces in deeper layers .
- Activation Function: To enable the network to learn complex, non-linear patterns, functions such as **ReLU** are used. These functions activate or deactivate neurons based on signal intensity, like biological brain activity.
- Pooling Layers: Layers such as **Max Pooling** are designed to reduce the image size (output dimensions) while preserving the most critical information (e.g., the pixel with the highest value in a specific region). This operation conserves computational resources and helps the network remain robust to small changes in object position or scale.

In our project, the use of CNNs is critical because the drone captures images from high altitudes. The network must identify the human body pattern even when the target is small, viewed from different angles, or situated under varying lighting conditions.

5.3 Object Detection

While standard "Image Classification" answers the question "What is in the image?" (e.g., "A dog"), **Object Detection** answers a more complex question: "What is in the image **and where** is it located?". In this task, the network is required to mark a **Bounding Box** around the object, typically defined by the center coordinates (x, y) along with the width and height of the box.

For a Search and Rescue (SAR) drone, this is the most crucial component: it is not enough to know that "there is a survivor in the frame"; it is mandatory to know their exact location within the image to derive the GPS coordinates for sending rescue teams.

5.4 The YOLO Algorithm (You Only Look Once)

To perform object detection in this project, we selected the YOLO architecture.

Unlike older methods that scanned the image thousands of times in different regions (Sliding Window), the YOLO algorithm "looks" at the entire image only once (hence its name).

How it works:

1. The system divides the input image into a grid.
2. For each grid cell, the network simultaneously predicts:
    a. Probability - does an object exist in this cell?
    b. What are the Bounding Box coordinates?
    c. What is the object class? (In our case: "Human").
3. At the end of the process, boxes with low probability are filtered out, leaving only the most accurate bounding box around the survivor.

The primary advantage of YOLO is its speed: the ability to process an image in a single pass allows for Real-Time operation, a necessary requirement for a UAV that flies and needs to make quick decisions or transmit data without latency.

5.5 Transfer Learning

Training a deep neural network from scratch requires massive datasets (millions of images) and significant computational power, which are not always available in student projects. Therefore, we utilized a technique called Transfer Learning. In this method, we take a network that has already been pre-trained on a vast, general dataset (such as ImageNet or COCO) and perform additional training (Fine-tuning) on our specific dataset (images from the simulation). The principle is that the initial layers of the network already

know how to identify "lines," "corners," and "textures" from the previous training. We "freeze" this knowledge and train only the later layers to identify our specific object—a human figure in a disaster zone. This approach allowed us to achieve high accuracy results in a relatively short time.

6. Building the Neural Network for Survivor Detection

6.1 Problem Definition and the Transition from Real to Synthetic Data

In this project, we addressed an Object Detection problem: the system receives an RGB image as input and must output the coordinates of any humans present in the frame.

The Challenge Encountered During the Project: In the initial planning phase (as described in the introduction), the vision was to deploy a physical drone in the real world. Based on this assumption, we began training the YOLO model using real-world drone imagery (the NOMAD dataset) so that the network would learn to identify humans in realistic field conditions.

However, as the project progressed, it became evident that constructing a physical drone with adequate processing capabilities and flight time was not feasible within the available resources. Consequently, a strategic decision was made to pivot the project to a Proof of Concept (PoC) within the AirSim simulation environment.

This transition introduced a new technological hurdle: the model trained on "real" images (NOMAD) failed to detect figures within the simulation. This failure was due to the Domain Gap - the visual discrepancy between the textures and lighting of the real world versus the computer-generated graphics of the simulator.

This realization required us to alter our training strategy and shift from using real-world data to Synthetic Data generated within the simulator, ensuring the model was adapted to the new testing environment.

6.2 Datasets

Reflecting the evolution of our solution, we utilized two types of datasets:

1. The Original Dataset: NOMAD (Real-World Data) An external dataset containing real aerial photographs of humans in open terrain.

- Original Purpose: To provide detection capabilities for a real-world mission.

- Outcome: Used for initial training but resulted in poor performance within the simulation due to the visual mismatch.

2. The Final Dataset: Custom AirSim Dataset (Synthetic Data) To bridge the domain gap, we created an independent dataset within our work environment.

- Data Collection: We flew the virtual drone inside the simulation generated images featuring human assets in various poses.
- Advantage: This dataset accurately represented the data the drone "sees" in real-time within the simulator, enabling the model to achieve high performance.

6.3 Training Process

The training process was executed using Transfer Learning in two main stages:

1. Pre-training: We started with a YOLOv8 model pre-trained on the COCO dataset (containing millions of general images). This stage endowed the network with the ability to recognize basic lines, contours, and shapes.

2. Fine-tuning: We performed additional training of the model using our Custom AirSim Dataset.
    - During this stage, we "froze" the initial layers of the network and trained only the final layers.
    - This process allowed the model to "adapt" to the unique textures of the simulation and learn to identify the computer-generated figures as humans, rather than searching for realistic features as found in NOMAD.

6.4 Results

The final model, trained on the simulation data, demonstrated a dramatic improvement compared to the initial attempts with NOMAD. We achieved a high accuracy (mAP@50) within the test environment, enabling the drone to identify survivors even when they were partially occluded or situated in shadows.

7. Conclusions

The development of this autonomous Search and Rescue (SAR) system yielded results and insights that evolved significantly from our initial vision. While the original objective was to deploy a physical drone (Crazyflie) for field detection, the transition to a high-fidelity simulation environment proved to be a strategic decision. This shift allowed us to focus on algorithmic robustness and solving complex computer vision challenges, independent of the hardware limitations that characterized the chosen drone.

The key achievements of the project include:

1. Implementation and Adaptation of a Simulation Environment: We utilized the Microsoft AirSim platform and adapted it to our project's needs by creating a dedicated disaster environment and importing relevant assets. The transition to a simulator was driven by the fact that the original drone's hardware did not meet the technical requirements (specifically camera quality and battery life) necessary for complex imaging and processing tasks. The simulation enabled us to bypass these constraints and validate the algorithmic feasibility of the system.

2. Overcoming the Domain Gap: A significant portion of our research addressed the disparity between real-world visual data (NOMAD) and the simulation environment. By generating a custom synthetic dataset and applying Transfer Learning techniques with the YOLOv8 model, we demonstrated how synthetic data can be effectively used to train Deep Learning models when real-world data is incompatible with the testing environment.

3. Implementation of a Complete End-to-End Workflow: We successfully implemented a full "Store-and-Process" architecture. The system demonstrated autonomous navigation capabilities (grid scanning), data acquisition, and offline processing to identify survivors with high accuracy.

The process highlighted the complexity of bridging the gap between theoretical AI models and their practical application, requiring engineering flexibility to shift from a hardware-centric solution to one based on software and simulation.

8. Future Work

Derived from the conclusions and the limitations of the current implementation, we propose two main directions for future development to bring the system closer to operational readiness:

1. On-Board Processing: Currently, the system operates in a "Store-and-Process" configuration (processing after landing). The logical next step is to deploy the optimized YOLO model onto a more powerful physical drone equipped with a dedicated embedded GPU (such as NVIDIA Jetson). This would enable Real-Time Inference, allowing the drone to alert rescue teams and transmit coordinates the moment a detection is made.
2. Advanced Path Planning and SLAM: Current navigation relies on a pre-defined GPS grid (Lawnmower pattern). Future iterations should integrate Simultaneous Localization and Mapping (SLAM) algorithms and reactive path planning. For instance, if the drone detects a potential survivor with low confidence, it should autonomously decide to descend for a closer inspection and verification before resuming its original scan path.